

Build Heap

1. Let $\text{index} = \text{length}/2 - 1$. This is the parent of the last node in the tree, i.e. $\text{list}[\text{index} + 1] \dots \text{list}[\text{length} - 1]$ are leaves
2. Convert the subtree with root of $\text{list}[\text{index}]$ into a heap.
 - a. Given $\text{list}[a]$ is root of tree, $\text{list}[b]$ is left child ($\text{root} * 2 + 1$), $\text{list}[c]$ is right child ($\text{root} * 2 + 2$), if exists
 - b. Compare $\text{list}[b]$ with $\text{list}[c]$ to determine larger child, $\text{list}[\text{largerIndex}]$
 - c. Compare $\text{list}[a]$ with $\text{list}[\text{largerIndex}]$. If $\text{list}[a] < \text{list}[\text{largerIndex}]$, then swap, else already a heap
 - d. If swap, repeat step 2 for the subtree of $\text{list}[\text{largerIndex}]$
3. Convert the subtree with the root of $\text{list}[\text{index} - 1]$ into a heap, repeat until $\text{list}[0]$

Heap Sort

1. Swap the root with the end of the list.
2. Heapify the list up to but not including the root
3. Repeat until there is only one node in the list

Show all steps

1. Make into a heap
2. Sort

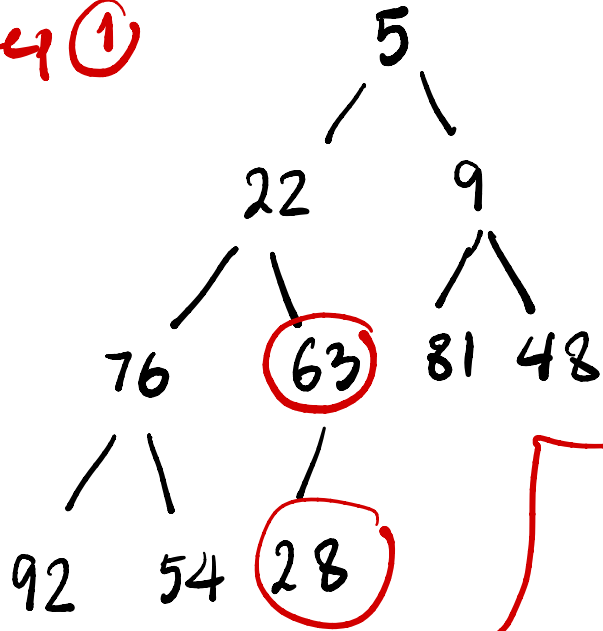
1. Make into a heap

2. Sort

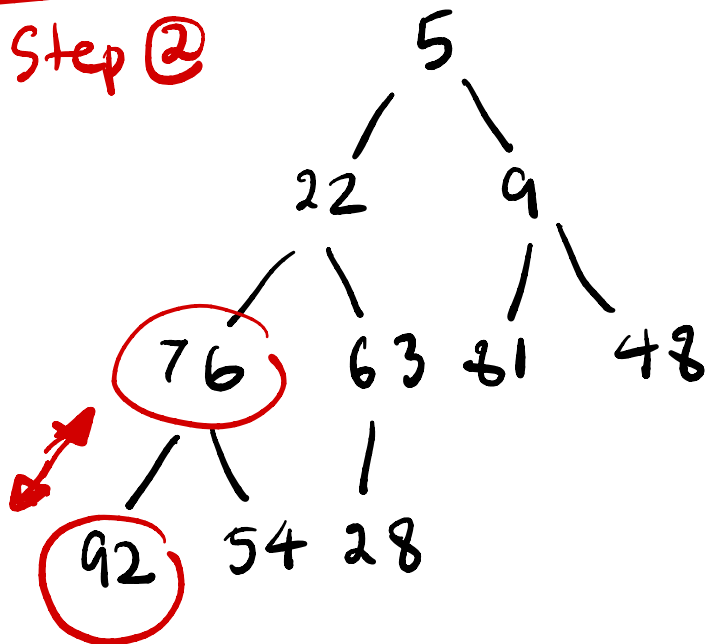
[illegible]

build max heap

step ①



Step ②



1. Find the last non leaf index

$$\text{index} = \frac{10}{2} - 1 = 4$$

2. check with their child

$63 > 28$ No swap

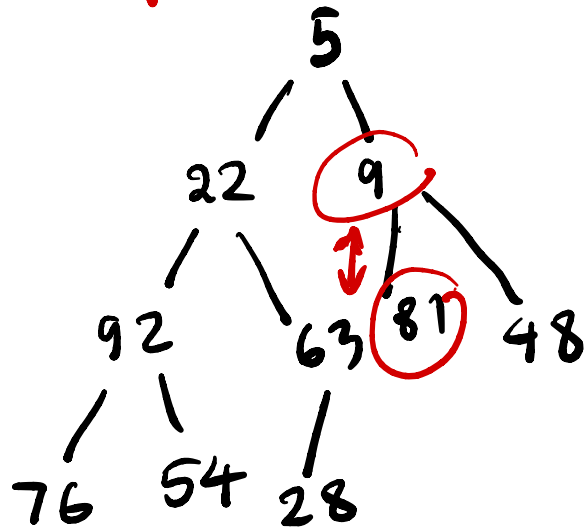
next check next index - 1

$$\text{index} = 3$$

$$\text{heap}[3] = 76$$

$76 < 92$, swap

Step 3

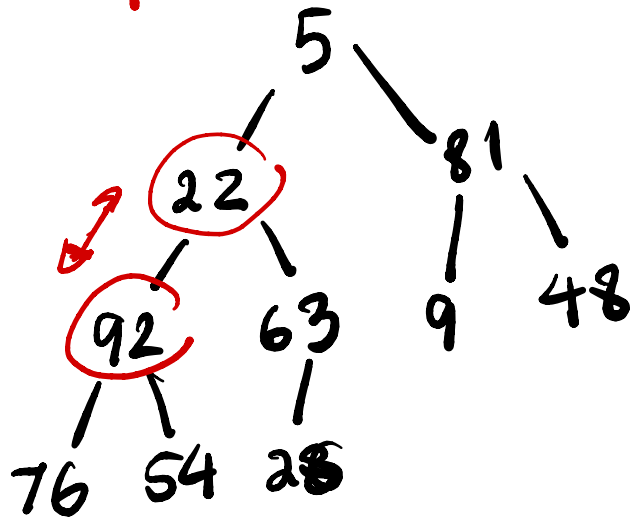


check index 2

heap[2] = 9

$9 < 81$, swap

Step 4



check index 1

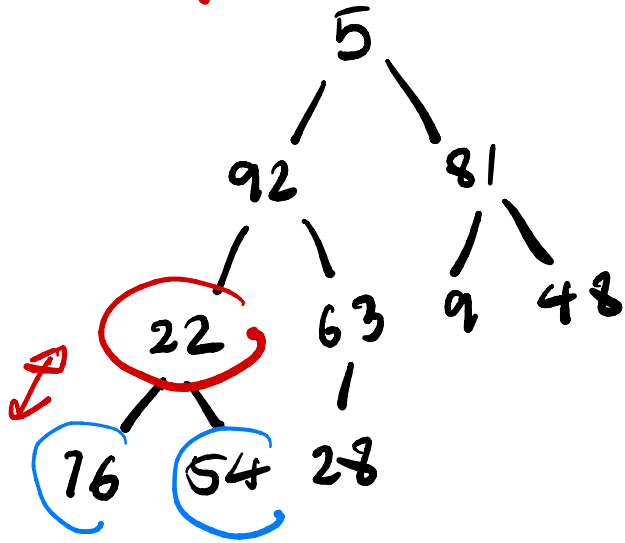
heap[1] = 22

Child index 3, 4

heap[3] = 92, heap[4] = 63

swap 22 with 92

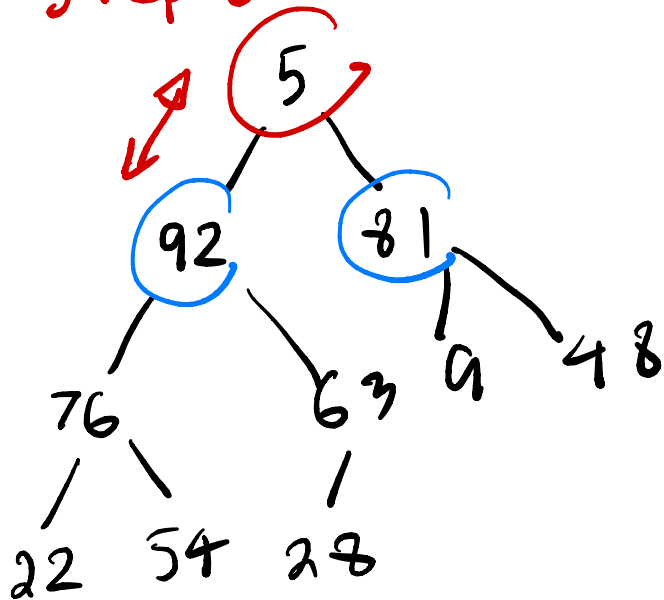
step 5



check index the largest child
check index 3
child Index, 7, 8

swap (22) with 76

step 6



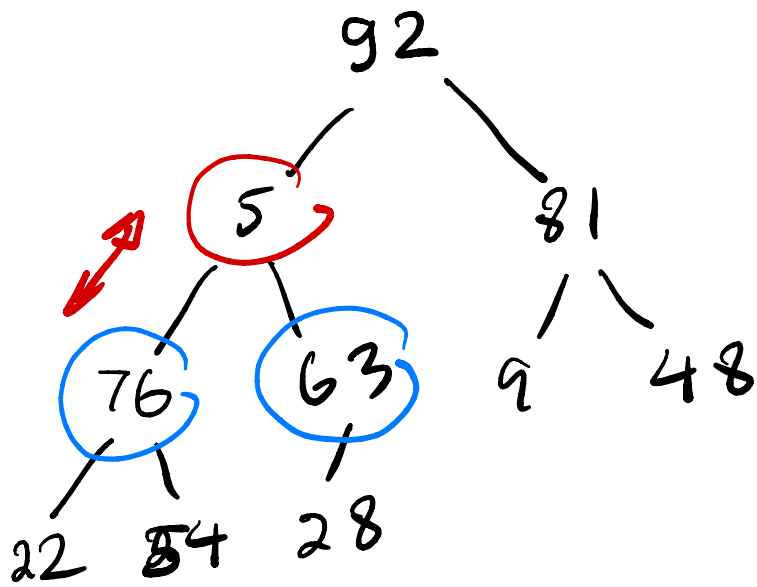
check index 0

child index 1, 2

swap with largest child index 1

swap (5) with 92

Step 7



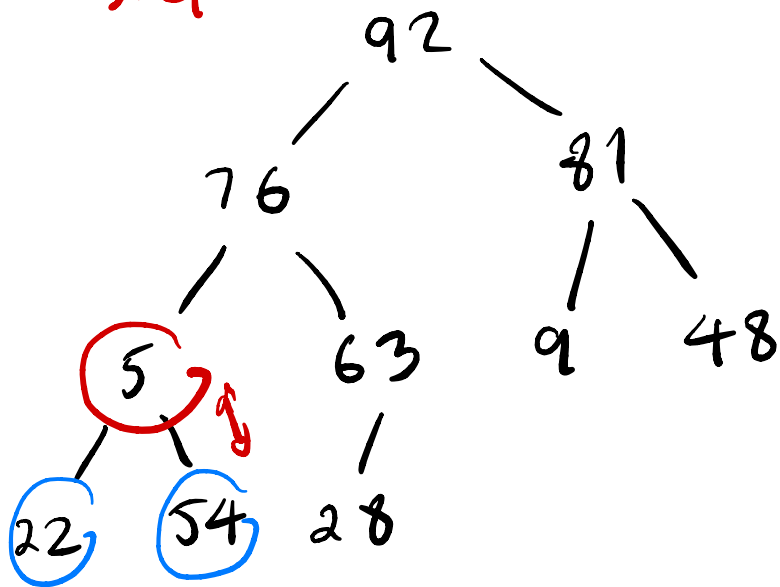
check index 1

child Index = 3, 4

swap (5) with (76)

next check the largest child Index
= 3

Step 8

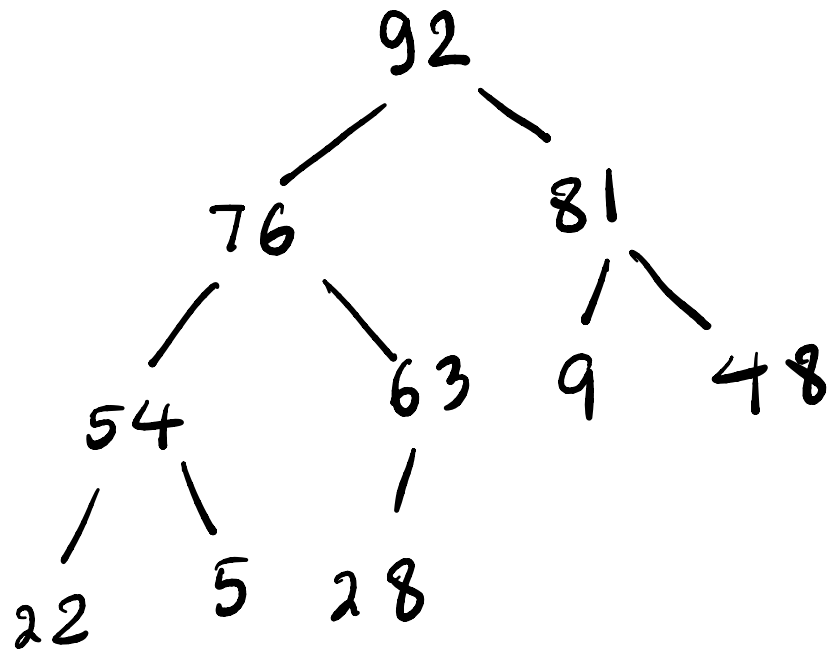


check index 3

child Index 7, 8

swap (5) with (54)

Final Max heap



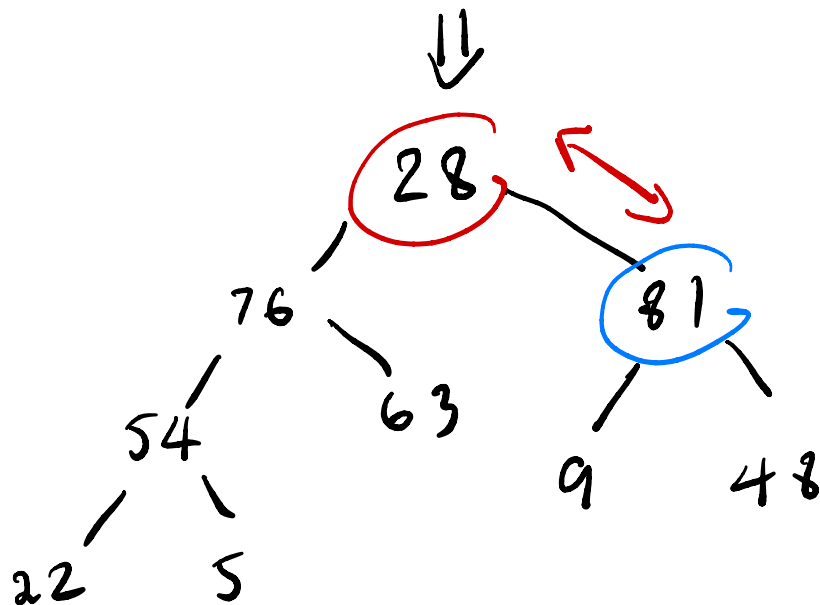
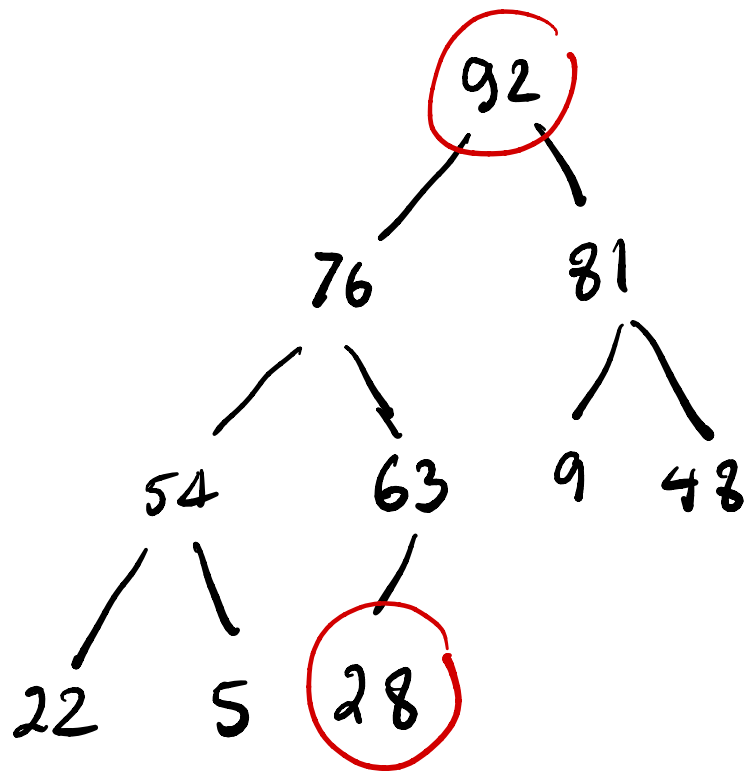
Max Heap

step 1 2 3 4 5 6 7 8

index= 4 3 2 1 3 0 1 3

[0]	5	5	5	5	5	92	92	92				
[1]	22	22	22	92	92	5	76	76				
[2]	9	9	81	81	81	81	81	81				
[3]	76	92	92	22	76	76	5	54				
[4]	63	63	63	63	63	63	63	63				
[5]	81	81	9	9	9	9	9	9				
[6]	48	48	48	48	48	48	48	48				
[7]	92	76	76	76	22	22	22	22				
[8]	54	54	54	54	54	54	54	5				
[9]	28	28	28	28	28	28	28	28				

Sorting



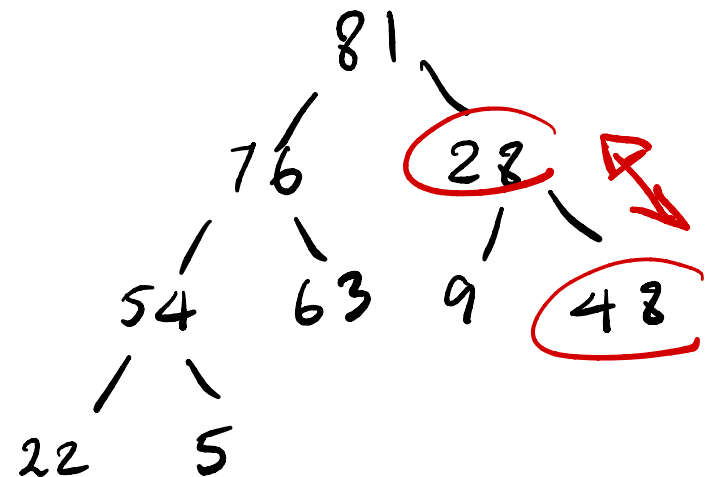
Step 1.

swap root with last node

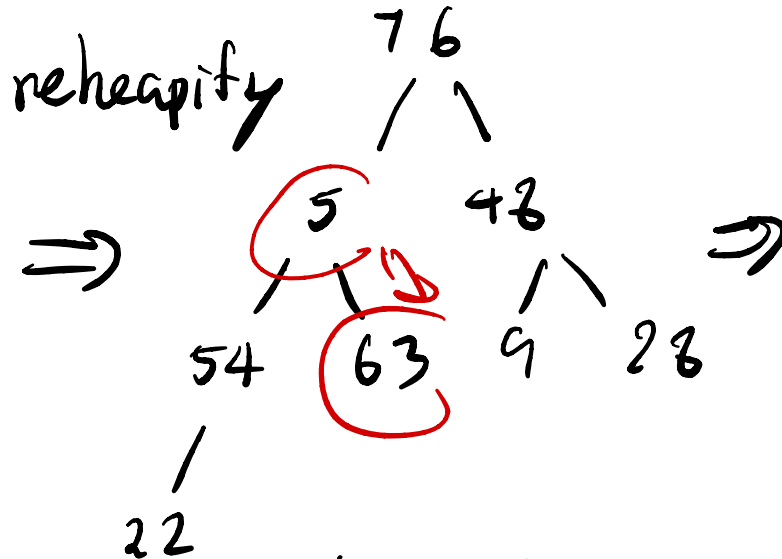
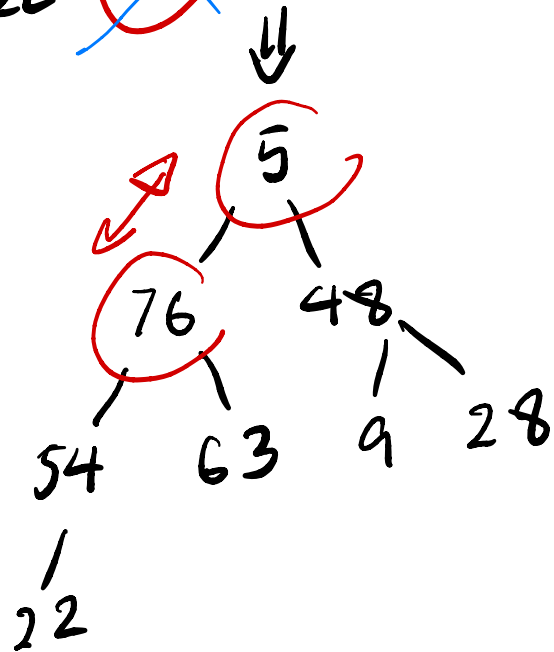
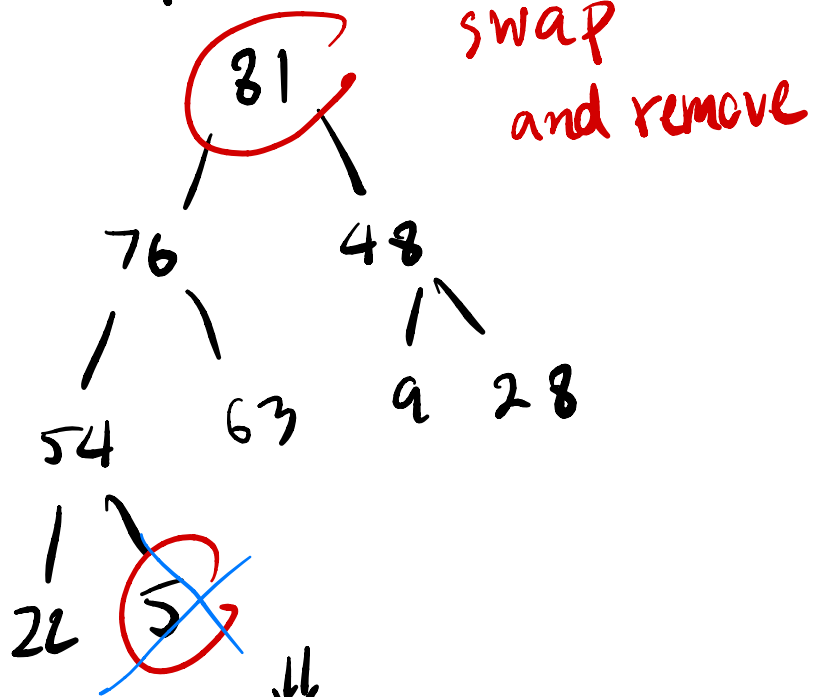
Then remove last node

→ reheapify

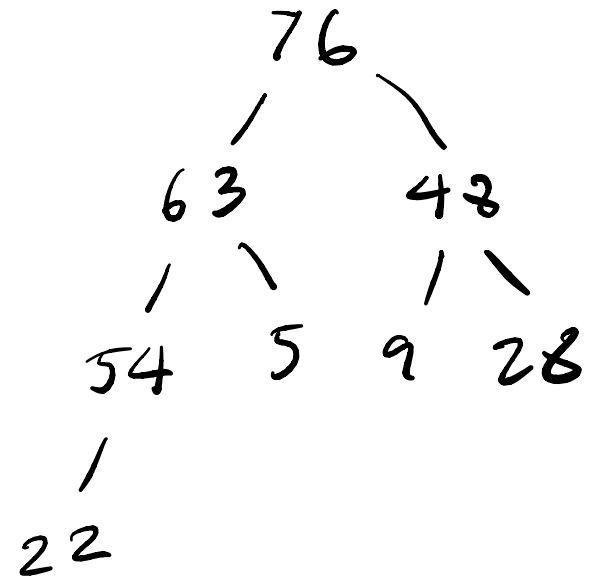
Step 1.1



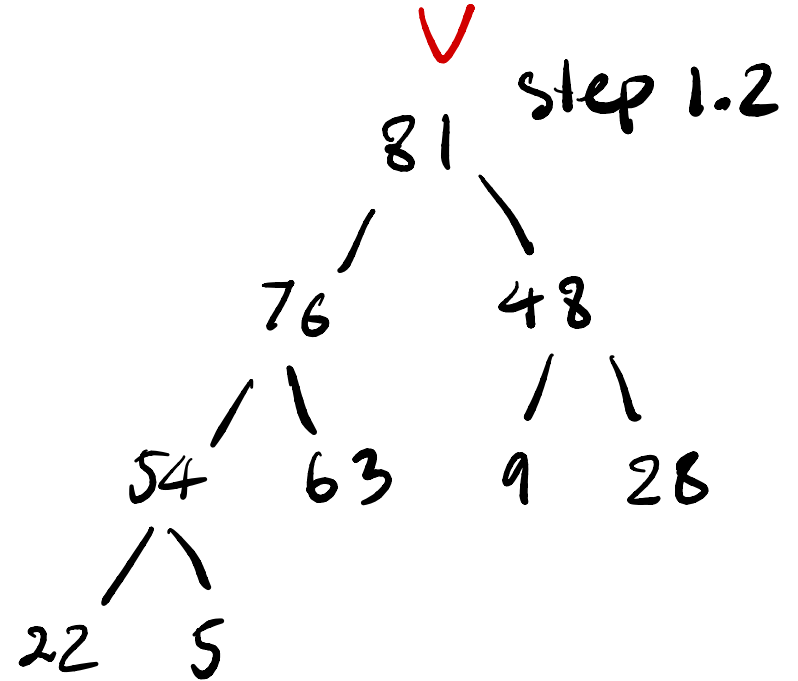
Step 2



Step 2.1

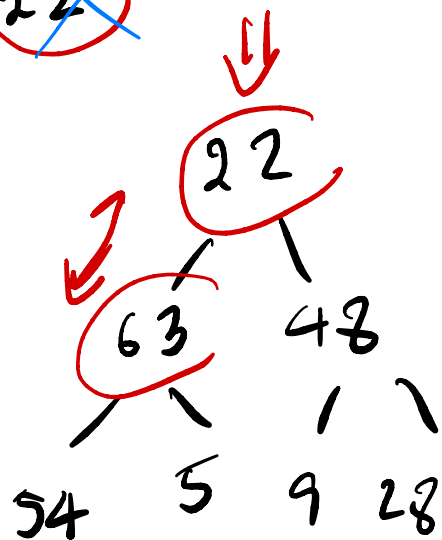
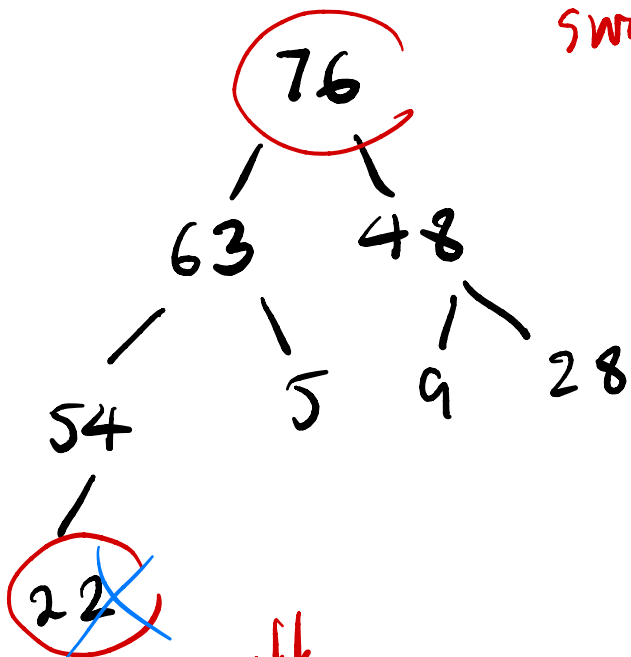


Step 2.2



step 3

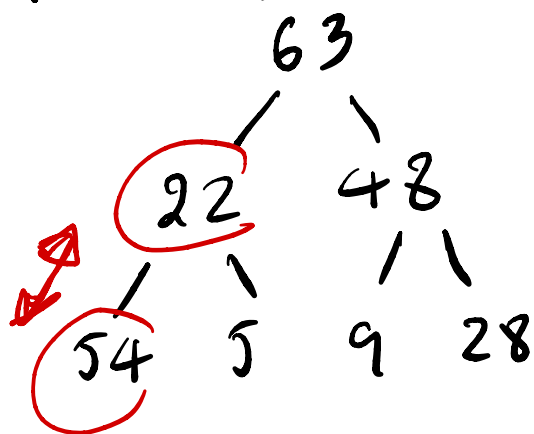
swap and remove



step 3.1

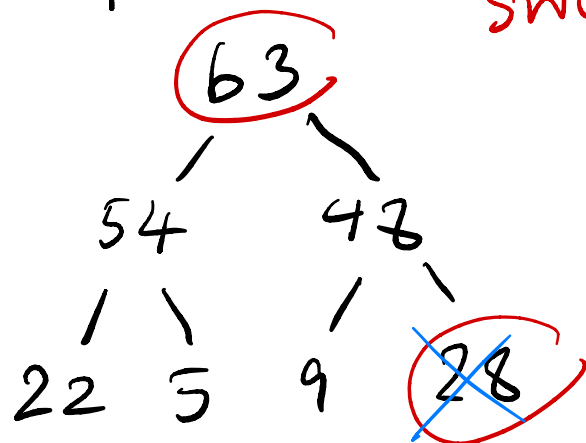
reheapify

⇒

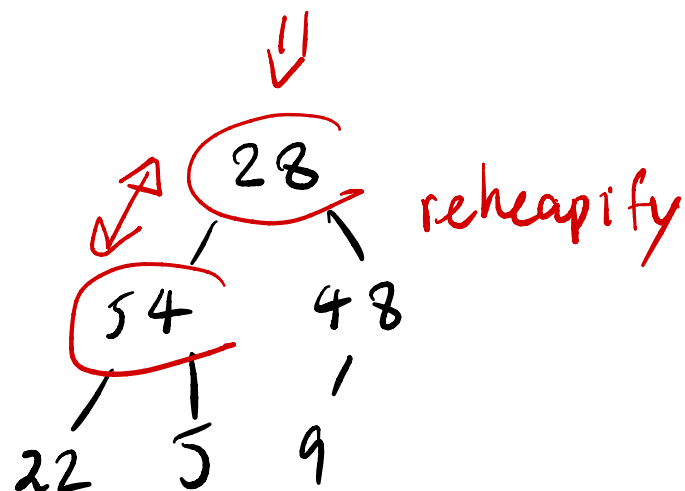


step 3.2

step 4

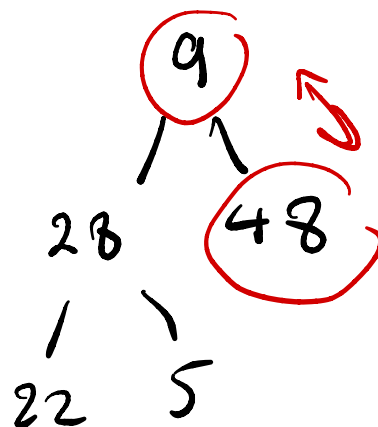


swap and remove



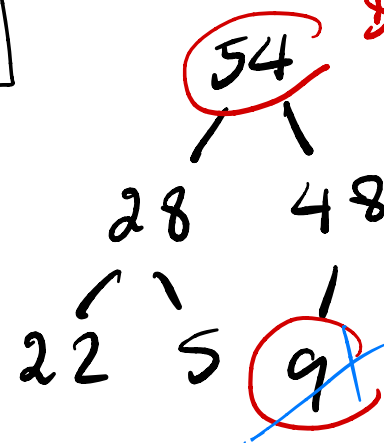
step 4.1

reheapify



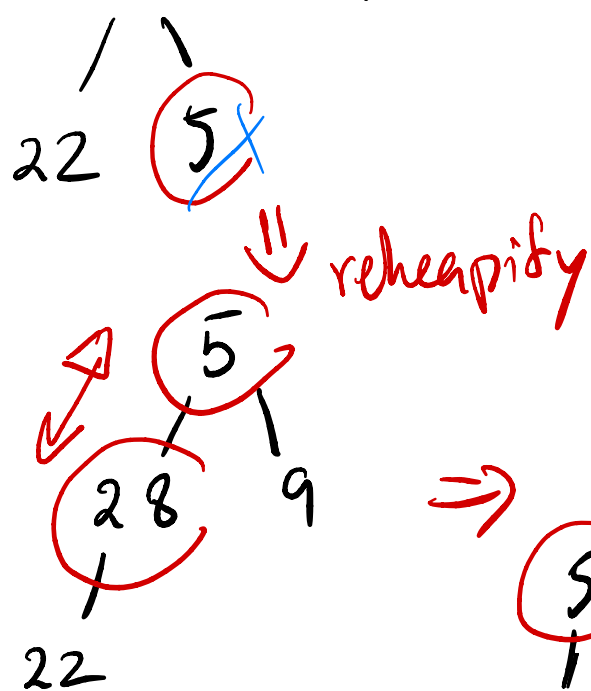
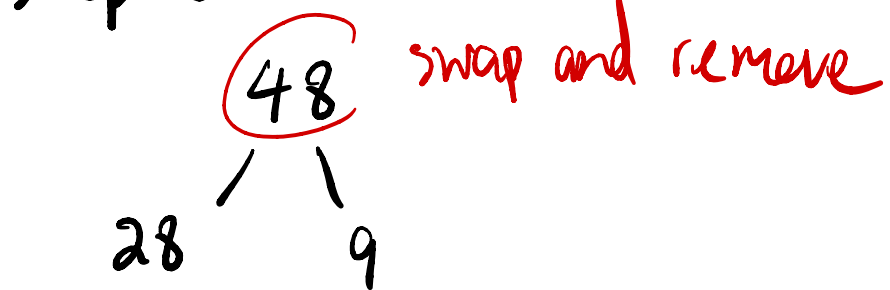
step 5.1

step 5

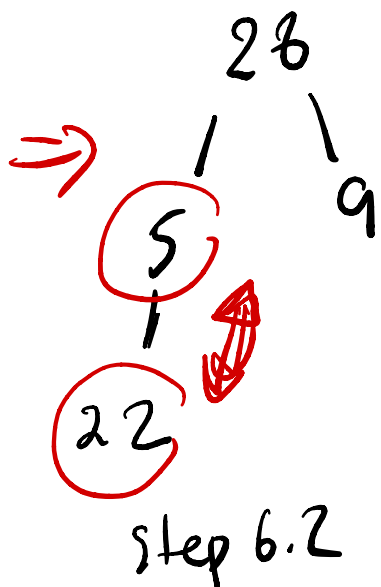


swap and remove

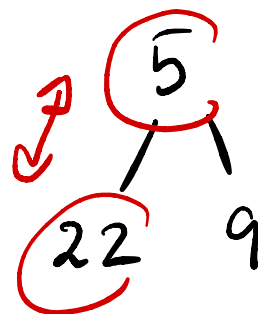
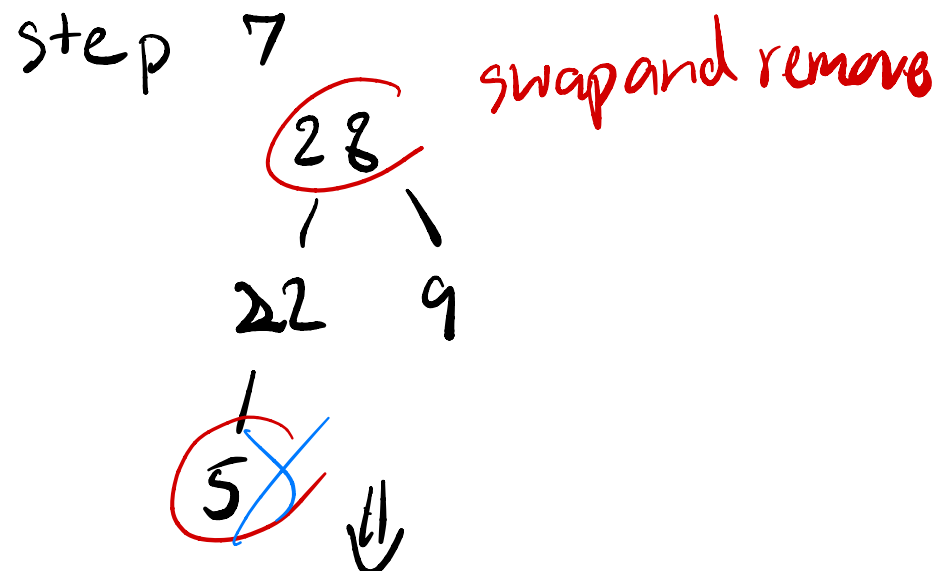
step 6



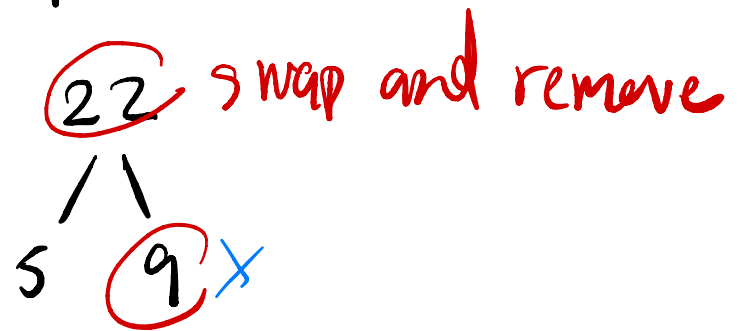
step 6.1



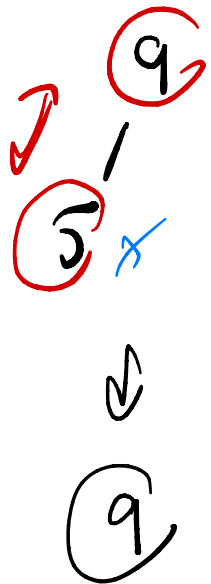
step 7



step 8



step 9



step

[0]

[1]

[2]

[3]

[4]

[5]

[6]

[7]

[8]

[9]

